MATERIAL COMPLEMENTARIO DE ESTUDIO

Next.js + PostgreSQL

Diseño y Desarrollo Web

Prácticas Profesionalizantes II

1. Cómo utilizar este material

El recorrido construye una aplicación pequeña pero completa. Cada etapa introduce un concepto de Next.js cuando se necesita y lo conecta con una funcionalidad verificable.

Objetivos de aprendizaje

- Comprender el App Router, las páginas y los layouts.
- Distinguir Server Components de Client Components.
- Conectar Next.js con PostgreSQL usando el paquete pg.
- Implementar Create, Read, Update y Delete (CRUD).
- Usar formularios HTML, Server Actions, validación, SQL parametrizado y revalidación.
- Comprobar la persistencia real de los datos.

Alcance

Se usa App Router, JavaScript y SQL directo mediante pg. La interfaz utiliza form, input, button, listas y etiquetas HTML nativas. No se utiliza una biblioteca de componentes ni se prioriza el diseño visual. Es un ejemplo didáctico, no una aplicación lista para producción.

Recorrido

Etapa	Resultado
1. Crear proyecto	Next.js inicia en desarrollo.
2. Rutas y layouts	Se comprende la estructura de archivos.
3. PostgreSQL	Existe la tabla tasks.
4. Capa de datos	Se centralizan conexión y consultas.
5. Read	La página lista tareas.
6. Create	El formulario agrega una tarea.
7. Update	Se modifica título y estado.
8. Delete	Se elimina una tarea.
9. Validación final	Los cambios persisten tras recargar.

2. Conceptos esenciales

React, Next.js y App Router

React es una biblioteca para construir interfaces. Next.js es un framework construido sobre React que agrega convenciones para rutas, renderizado en servidor y lógica de aplicación. App Router organiza las rutas mediante archivos dentro de app/. Una carpeta representa un segmento de URL; page.js define la interfaz de esa ruta y layout.js define una estructura compartida.

Archivo	Responsabilidad
app/layout.js	Layout raíz compartido y estructura HTML.
app/page.js	Página principal (/): consulta y presenta tareas.
app/actions.js	Funciones de servidor que modifican datos.
lib/db.js	Pool de conexiones PostgreSQL.
lib/tasks.js	Consultas SQL de tareas.
.env.local	Cadena de conexión local; no debe subirse al repositorio.

Server Components y Client Components

En App Router, los componentes son Server Components por defecto. Se ejecutan en el servidor y pueden obtener datos antes de renderizar. Los Client Components se marcan con "use client" y se utilizan cuando hacen falta estado local, efectos o APIs del navegador. En este ejercicio, la lectura se resuelve en un Server Component y los formularios HTML pueden invocar Server Actions sin convertirse por eso en Client Components.

> La conexión a PostgreSQL y sus credenciales deben permanecer en el servidor. Nunca se debe importar un módulo de base de datos en un Client Component.

CRUD

Operación	Qué hace el ejemplo
Create	Agregar una tarea.
Read	Listar tareas guardadas.
Update	Editar título y cambiar estado.
Delete	Eliminar una tarea.

3. Crear el proyecto

Requisitos: Node.js compatible con la versión instalada de Next.js, PostgreSQL accesible, editor y terminal.

```
npx create-next-app@latest todo-next
cd todo-next
npm run dev
npm install pg
```

Al ejecutar create-next-app, aceptar la configuración recomendada y seleccionar JavaScript si se ofrece la opción. Este documento asume app/ en la raíz. Si el proyecto utiliza src/app/, ubicar allí los archivos y ajustar las rutas de importación. Abrir http://localhost:3000 para comprobar el servidor.

4. Preparar PostgreSQL

Crear una base llamada todo_app o usar una base existente. Ejecutar este SQL en ella:

```
CREATE TABLE IF NOT EXISTS tasks (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  completed BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT tasks_title_not_blank
    CHECK (LENGTH(TRIM(title)) > 0)
);

INSERT INTO tasks (title, completed)
VALUES ('Estudiar App Router', FALSE),
       ('Conectar PostgreSQL', TRUE);
```

id identifica cada fila; title almacena el texto; completed representa el estado; created_at registra la creación. La restricción evita títulos vacíos o compuestos solo por espacios.

Configurar la conexión

Crear .env.local en la raíz del proyecto y adaptar los valores al entorno:

```
DATABASE_URL=postgresql://usuario:contraseña@localhost:5432/todo_app
```

No subir credenciales reales al repositorio. Confirmar que .env.local esté ignorado por Git. Si se cambia una variable de entorno, reiniciar el servidor de desarrollo. En algunos proveedores pueden ser necesarios parámetros SSL específicos.

5. Centralizar conexión y consultas

Paso 1: pool de conexiones

Crear lib/db.js. Un pool administra conexiones reutilizables a la base.

```
import "server-only";
import pg from "pg";

const { Pool } = pg;

if (!process.env.DATABASE_URL) {
  throw new Error("Falta configurar DATABASE_URL");
```

```
}

const globalForPg = globalThis;
const pool = globalForPg.pgPool ??
  new Pool({ connectionString: process.env.DATABASE_URL });

if (process.env.NODE_ENV !== "production") {
  globalForPg.pgPool = pool;
}

export default pool;
```

server-only ayuda a detectar importaciones accidentales desde código de cliente. Reutilizar el pool evita crear repetidamente nuevas conexiones durante la recarga automática de desarrollo. En despliegues serverless, la estrategia de conexión debe ajustarse al proveedor.

Paso 2: operaciones SQL

Crear lib/tasks.js para mantener las consultas fuera de las páginas y acciones.

```
import "server-only";
import pool from "./db.js";

export async function getTasks() {
  const result = await pool.query(
    `SELECT id, title, completed, created_at
     FROM tasks ORDER BY created_at DESC, id DESC`
  );
  return result.rows;
}

export async function createTask(title) {
  const result = await pool.query(
    `INSERT INTO tasks (title) VALUES ($1)
     RETURNING id, title, completed, created_at`,
    [title]
  );
  return result.rows[0];
}

export async function updateTask(id, title, completed) {
  const result = await pool.query(
    `UPDATE tasks SET title = $1, completed = $2
     WHERE id = $3
     RETURNING id, title, completed, created_at`,
    [title, completed, id]
  );
  return result.rows[0] ?? null;
}

export async function deleteTask(id) {
  const result = await pool.query(
    `DELETE FROM tasks WHERE id = $1 RETURNING id`,
    [id]
  );
  return result.rowCount === 1;
}
```

Los marcadores $1, $2 y $3 separan la instrucción SQL de los valores. Esto reduce el riesgo de inyección SQL frente a concatenar directamente texto ingresado. Los parámetros no reemplazan la validación de entradas.

¿Por qué una capa de datos?

- La conexión se configura en un único lugar.
- Las funciones expresan claramente su intención.
- Las páginas y acciones reutilizan las mismas consultas.
- Es más sencillo mantener y probar el código.

6. Read: listar las tareas

Reemplazar app/page.js por este ejemplo. La página es un Server Component asíncrono y espera la consulta antes de renderizar.

```
import { getTasks } from "../lib/tasks.js";

export default async function HomePage() {
  const tasks = await getTasks();

  return (
```

```
    <main>
      <h1>Lista de tareas</h1>
      <p>Las tareas se almacenan en PostgreSQL.</p>
      {tasks.length === 0 ? (
        <p>Todavía no hay tareas.</p>
      ) : (
        <ul>
          {tasks.map((task) => (
            <li key={task.id}>
              {task.completed ? "✓" : "■"} {task.title}
            </li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

- async/await permite esperar la consulta.
- map transforma cada fila en un elemento de la lista.
- key={task.id} ofrece a React una clave estable.
- La condición presenta un mensaje si no hay registros.
- No hace falta useEffect ni fetch desde el navegador para esta lectura inicial.

Flujo de lectura

```
Navegador solicita /
  → app/page.js se ejecuta en el servidor
  → getTasks() ejecuta SELECT
  → PostgreSQL devuelve filas
  → Next.js renderiza la página
```

Prueba: abrir la página y verificar que aparezcan las dos tareas de ejemplo.

7. Create: agregar una tarea

¿Qué es una Server Action?

Una Server Action es una función ejecutada en el servidor que puede recibir datos de una interacción, como un formulario. Permite implementar esta mutación sin crear una API REST separada. Todo dato recibido debe validarse en el servidor.

Paso 1: crear app/actions.js

```
"use server";

import { revalidatePath } from "next/cache";
import {
  createTask as insertTask,
  updateTask as saveTask,
  deleteTask as removeTask,
} from "../lib/tasks.js";

function readTitle(formData) {
  const title = String(formData.get("title") ?? "").trim();
  if (!title || title.length > 200) {
    throw new Error("El título debe tener entre 1 y 200 caracteres.");
  }
  return title;
}

function readId(formData) {
  const id = String(formData.get("id") ?? "");
  if (!/^\d+$/.test(id)) throw new Error("Identificador inválido.");
  return id;
}

export async function createTaskAction(formData) {
  const title = readTitle(formData);
  await insertTask(title);
  revalidatePath("/");
}

export async function updateTaskAction(formData) {
  const id = readId(formData);
  const title = readTitle(formData);
  const completed = formData.get("completed") === "on";
  const row = await saveTask(id, title, completed);
  if (!row) throw new Error("La tarea no existe.");
  revalidatePath("/");
```

```
}

export async function toggleTaskAction(formData) {
  const id = readId(formData);
  const title = readTitle(formData);
  const completed = formData.get("completed") === "true";
  const row = await saveTask(id, title, completed);
  if (!row) throw new Error("La tarea no existe.");
  revalidatePath("/");
}

export async function deleteTaskAction(formData) {
  const id = readId(formData);
  await removeTask(id);
  revalidatePath("/");
}
```

La directiva "use server" declara acciones de servidor. readTitle valida y normaliza el título; readId valida el identificador. En una app real, es preferible mostrar errores de validación de manera controlada en la interfaz. Si hay usuarios, cada acción también debe comprobar identidad y autorización.

Paso 2: conectar un formulario HTML

Actualizar app/page.js para importar la acción y añadir el formulario de alta. En esta primera etapa, la lista puede mantenerse como en la sección anterior.

```
import { getTasks } from "../lib/tasks.js";
import { createTaskAction } from "./actions.js";

export default async function HomePage() {
  const tasks = await getTasks();
  return (
    <main>
      <h1>Lista de tareas</h1>
      <form action={createTaskAction}>
        <label htmlFor="new-title">Nueva tarea</label>
        <input id="new-title" name="title" required maxLength={200} />
        <button type="submit">Agregar</button>
      </form>
      {tasks.length === 0 ? <p>Todavía no hay tareas.</p> : (
        <ul>{tasks.map((task) => (
          <li key={task.id}>
            {task.completed ? "✔" : "■"} {task.title}
          </li>
        ))}</ul>
      )}
    </main>
  );
}
```

name="title" es la clave que la acción recupera con formData.get("title"). required y maxLength mejoran la experiencia, pero no sustituyen la validación de servidor.

¿Qué hace revalidatePath?

Después de insertar, revalidatePath("/") solicita revalidar la ruta principal para que la lista vuelva a reflejar los datos actualizados.

- Agregar una tarea.
- Verificar que aparece en la lista.
- Recargar la página y comprobar que sigue apareciendo.
- Probar un título vacío.

8. Update y Delete: completar el CRUD

Reemplazar app/page.js por esta versión completa. Mantiene la lectura y el alta e incorpora un formulario para editar título y estado, un botón de cambio rápido de estado y un formulario de eliminación.

```
import { getTasks } from "../lib/tasks.js";
import {
  createTaskAction,
  updateTaskAction,
  toggleTaskAction,
  deleteTaskAction,
} from "./actions.js";

export default async function HomePage() {
  const tasks = await getTasks();
```

```
  return (
    <main>
      <h1>Lista de tareas</h1>
      <form action={createTaskAction}>
        <label htmlFor="new-title">Nueva tarea</label>
        <input id="new-title" name="title" required maxLength={200} />
        <button type="submit">Agregar</button>
      </form>

      {tasks.length === 0 ? <p>Todavía no hay tareas.</p> : (
        <ul>
          {tasks.map((task) => (
            <li key={task.id}>
              <form action={updateTaskAction}>
                <input type="hidden" name="id" value={task.id} />
                <label>
                  Tarea
                  <input name="title" defaultValue={task.title}
                    required maxLength={200} />
                </label>
                <label>
                  Completada
                  <input type="checkbox" name="completed"
                    defaultChecked={task.completed} />
                </label>
                <button type="submit">Guardar cambios</button>
              </form>

              <form action={toggleTaskAction}>
                <input type="hidden" name="id" value={task.id} />
                <input type="hidden" name="title" value={task.title} />
                <input type="hidden" name="completed"
                  value={String(!task.completed)} />
                <button type="submit">
                  {task.completed ? "Marcar pendiente" : "Marcar completada"}
                </button>
              </form>

              <form action={deleteTaskAction}>
                <input type="hidden" name="id" value={task.id} />
                <button type="submit">Eliminar</button>
              </form>
            </li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

Qué sucede en una actualización

- El formulario envía el id y los campos modificados.
- La Server Action valida los datos recibidos.
- La capa de datos ejecuta UPDATE parametrizado.
- RETURNING permite detectar si se actualizó una fila.
- revalidatePath("/") hace que la página muestre el resultado actualizado.

Los campos ocultos facilitan enviar valores, pero no son confiables: pueden modificarse. En una aplicación multiusuario, el servidor debe comprobar que el registro pertenece a la persona autorizada.

Qué sucede al eliminar

deleteTaskAction obtiene el id, llama a removeTask, que ejecuta DELETE parametrizado, y revalida la ruta. Para una aplicación real, se recomienda confirmar la eliminación y presentar errores de forma controlada.

Pruebas de aceptación

- Cambiar el título y guardar.
- Marcar y desmarcar Completada.
- Usar el botón de cambio rápido de estado.
- Eliminar una tarea.
- Recargar después de cada operación y comprobar que los cambios se mantengan.

9. Layout raíz y estructura final

app/layout.js define la estructura compartida de la aplicación. El archivo creado por Next.js puede simplificarse así, conservando otros metadatos si se desea:

```
export const metadata = {
  title: "Lista de tareas",
  description: "Ejemplo CRUD con Next.js y PostgreSQL",
};

export default function RootLayout({ children }) {
  return (
    <html lang="es">
      <body>{children}</body>
    </html>
  );
}
```

children representa la página o el contenido de la ruta. El layout raíz es el lugar para definir el HTML compartido. No se necesita CSS para demostrar el flujo de datos.

```
todo-next/
■■■ app/
■   ■■■ actions.js
■   ■■■ layout.js
■   ■■■ page.js
■■■ lib/
■   ■■■ db.js
■   ■■■ tasks.js
■■■ .env.local
■■■ package.json
■■■ ...
```

Si el proyecto usa src/app, ubicar los archivos de forma consistente y ajustar las importaciones.

10. Flujo end-to-end

```
LECTURA:
Navegador → app/page.js (Server Component)
 → getTasks() → pool.query(SELECT) → PostgreSQL
 ← filas → HTML renderizado

ESCRITURA:
Formulario HTML → Server Action
 → validación → función en lib/tasks.js
 → SQL parametrizado → PostgreSQL
 → revalidatePath("/") → lista actualizada
```

El navegador no se conecta directamente a PostgreSQL. Envía datos al servidor; el servidor valida y ejecuta SQL, y la página vuelve a renderizarse con el estado persistido.

Comprobación	Qué demuestra
La tarea aparece tras el envío	El flujo de formulario y acción funciona.
Sigue tras recargar	La lectura recupera un dato persistido, no solo estado local.
Sigue tras reiniciar Next.js	No dependía solo de memoria del proceso.
SQL muestra el cambio	La mutación llegó a PostgreSQL.

11. Diagnóstico y seguridad básica

Síntoma	Qué revisar
ECONNREFUSED	PostgreSQL activo, host, puerto y acceso desde el entorno de Next.js.
Error de autenticación	Usuario y contraseña de DATABASE_URL.
Base o tabla inexistente	Nombre de base y ejecución del SQL en la base correcta.
DATABASE_URL undefined	Nombre exacto, ubicación de .env.local y reinicio del servidor.
Campo no recibido	Atributo name del input y clave leída por FormData.
Cambios no visibles	Que se llame revalidatePath("/") después de la mutación.
Error al importar pg	Verificar npm install pg y que el módulo solo se use en servidor.

- No exponer DATABASE_URL mediante variables NEXT_PUBLIC_*.
- Validar en servidor todos los datos recibidos.
- Usar consultas parametrizadas.
- No importar el módulo de base de datos en Client Components.
- En apps con cuentas, verificar autorización dentro de cada acción.
- No mostrar credenciales ni detalles internos al usuario final.

12. Actividad integradora

Implementar y demostrar una aplicación TODO que cumpla estos criterios. Se evalúan integración, lógica y comprensión del flujo, no la estética.

- Crear una tabla con id, título, estado y fecha de creación.
- Conectar Next.js con PostgreSQL mediante variable de entorno y pool.
- Listar tareas desde un Server Component.
- Crear tareas mediante formulario HTML y Server Action.
- Actualizar el título de una tarea.
- Cambiar entre pendiente y completada.
- Eliminar una tarea.
- Validar el título en servidor y usar SQL parametrizado.
- Revalidar la ruta después de cada mutación.
- Demostrar persistencia tras recargar.

Criterios de aceptación

Criterio	Evidencia
Read	Se listan filas existentes en PostgreSQL.
Create	Una nueva tarea persiste tras recargar.
Update	Título y estado cambian en la base.
Delete	La tarea desaparece de la lista y la tabla.
Arquitectura	Se distinguen layout, página, acciones y capa de datos.
Seguridad básica	Credenciales privadas, validación de servidor y SQL parametrizado.

14. Glosario

Término	Definición
App Router	Sistema de rutas basado en archivos dentro de app/.
Layout	Estructura compartida que envuelve páginas.
Server Component	Componente ejecutado en el servidor por defecto en App Router.
Client Component	Componente que habilita interacción y APIs del navegador; se marca con use client.
PostgreSQL	Sistema gestor de base de datos relacional.
Pool	Conjunto reutilizable de conexiones a la base.
SQL parametrizado	Consulta que separa instrucción y valores.
Server Action	Función de servidor invocada desde una interacción.
FormData	Representación de los campos enviados por un formulario.
Revalidación	Actualización de los datos renderizados o almacenados en caché de una ruta.
Persistencia	Conservación de datos más allá de una petición o recarga.

15. Referencias oficiales

- Next.js — App Router: https://nextjs.org/docs/app
- Next.js — Server and Client Components: https://nextjs.org/docs/app/getting-started/server-and-client-components
- Next.js — Mutating Data y Server Actions: https://nextjs.org/docs/app/getting-started/mutating-data
- Next.js — Fetching Data: https://nextjs.org/docs/app/getting-started/fetching-data
- node-postgres — Queries y parámetros: https://node-postgres.com/features/queries
- PostgreSQL — Documentación oficial: https://www.postgresql.org/docs/